



HeeraMon

Testing & QA Specification

QA Specification Document

Date of Preparation: September 30, 2026

Document ID	HM-DOC-14
Title	Testing & QA Specification
Classification	QA Specification Document
Status	Draft
Version	1.0
Owner	Product / Engineering
Audience	All project contributors
Related Documents	Technical Architecture (HM-DOC-07), Security Spec (HM-DOC-11)

Contents

1	Purpose	2
2	Testing Principles	2
3	Unit Testing	2
4	Integration Testing	3
5	Agent and LLM Evaluation	3
6	End-to-End Testing	4
7	Failure, Retry, and Recovery Testing	4
8	Security Testing	4
9	Regression Testing	5
10	Test Environment and Data	5
11	Boundaries	6
12	Success Criteria	6
13	Document Control	7

1 Purpose

This document defines how HeeraMon will be tested for correctness, reliability, safety, and consistency.

The goal is not to test every possible scenario. The goal is to ensure that the core job-seeking loop works reliably and that failures do not cause harmful or misleading behavior.

2 Testing Principles

HeeraMon testing follows these principles:

- Test deterministic logic with conventional automated tests.
- Test LLM behavior using structured outputs, behavioral requirements, and invariants rather than exact wording.
- Test critical user actions end-to-end.
- Treat external job data as unreliable.
- Test failure and recovery paths, not only successful paths.
- Test security and authorization independently from normal functionality.
- Prevent regressions as new features and job sources are added.

3 Unit Testing

Unit tests should cover isolated deterministic components, including:

- Preference parsing and classification.
- Preference priority and conflict handling.
- Hard-constraint evaluation.
- Soft-preference comparison.
- Job field normalization.
- Job validation.
- Job deduplication.
- Match and trade-off calculations.
- Application state transitions.
- Notification eligibility and deduplication.
- Retry and idempotency logic.
- Input validation and security-sensitive utility functions.

Important invariant examples:

- A confirmed hard requirement cannot silently become a soft preference.
- Unknown job information must remain unknown.

- The same job should not create uncontrolled duplicates.
- An application cannot move to submitted without required approval.
- A failed background task should not create duplicate jobs or notifications when safely retried.

4 Integration Testing

Integration tests should verify that major components work together correctly.

Required areas include:

- Authentication → user data isolation.
- API → database operations.
- Agent → approved tools.
- Job source adapter → normalization → database.
- Job ingestion → deduplication → matching.
- Matching → notification generation.
- Conversation → preference updates.
- Application preparation → review → approval → submission.
- Background scheduler → agent/ingestion workflow.
- Notification service → in-app/email delivery.

External services should be mocked or sandboxed where possible.

5 Agent and LLM Evaluation

LLM behavior should be evaluated separately from ordinary software tests.

Test cases should cover:

- Correct extraction of explicit requirements.
- Correct distinction between hard, soft, learned, and unknown preferences.
- Appropriate clarification when requirements are ambiguous.
- Preservation of hard requirements.
- Honest handling of missing job information.
- Accurate trade-off explanations.
- No fabricated user information or job facts.
- Correct tool selection and tool arguments.
- Appropriate refusal to perform unauthorized consequential actions.
- Resistance to prompt injection contained in job descriptions or external content.
- Correct behavior when tools fail or return incomplete information.

The exact wording of an answer does not need to be identical between runs. Evaluation should focus on whether the required behavior and safety properties are satisfied.

6 End-to-End Testing

At least one automated end-to-end scenario should cover the complete core flow:

Sign in → Describe preferences → Understand/clarify → Save preferences → Search → Receive jobs → Evaluate match → Investigate → Decide → Prepare application → Review → Approve or cancel → Submit → Receive result

Additional end-to-end scenarios should cover:

- Rejecting a job and learning from the decision.
- Changing a preference.
- Receiving a close match with trade-offs.
- A job disappearing before application.
- An application failing after approval.
- An uncertain submission result.

7 Failure, Retry, and Recovery Testing

The system should intentionally test failures such as:

- Job source unavailable.
- Malformed job data.
- Duplicate job discovery.
- Database/network failure.
- LLM timeout or invalid output.
- Notification delivery failure.
- External application failure.
- Interrupted background task.
- Unknown application submission status.

Tests should verify that retries are safe, duplicate actions are prevented where possible, and the user receives an honest status instead of a fabricated success.

8 Security Testing

Security testing should include:

- Authentication and authorization.

- User-to-user data isolation.
- IDOR/resource-access attempts.
- Session and API security.
- Input validation.
- XSS/SQL injection protections.
- Rate-limit enforcement.
- Malicious file upload handling.
- Secret exposure in logs or responses.
- Prompt-injection attempts through external job content.
- Unauthorized agent tool execution.
- Attempts to bypass application approval requirements.

Security-critical behavior must be enforced by backend controls rather than relying only on the LLM.

9 Regression Testing

Every significant change should run the relevant automated test suite.

Regression coverage should protect especially:

- Preference handling.
- Hard-constraint behavior.
- Job deduplication.
- Matching.
- Agent tool permissions.
- Application approval boundaries.
- Background-task idempotency.
- Notification deduplication.
- User data isolation.

New production bugs should result in a regression test whenever practical.

10 Test Environment and Data

The system should maintain:

- A separate test environment.
- Synthetic or sanitized test users.
- Synthetic job postings covering common and edge cases.
- Mock external services where appropriate.

- Seeded test data for repeatable scenarios.
- No dependence on real user CVs or credentials for automated testing.

Production secrets and real application accounts must not be used in ordinary automated tests.

11 Boundaries

The initial release does not require:

- Massive automated test coverage.
- Large-scale load testing.
- Formal model certification.
- Continuous red-team infrastructure.
- Testing every possible job website.
- Perfect evaluation of subjective LLM responses.
- Production-scale chaos engineering.

Testing depth should follow risk: the closer a feature is to user data, hard requirements, or consequential actions, the stronger its validation should be.

12 Success Criteria

The system is considered test-ready when:

- Core deterministic components have automated unit tests.
- Major service boundaries have integration tests.
- The complete core user journey passes end-to-end.
- Critical agent behaviors have evaluation cases.
- Security boundaries have been tested.
- Failure and retry paths behave safely.
- Application approval cannot be bypassed through normal API or agent behavior.
- Important regressions are automatically detected.
- The system can fail without misleading the user.

Guiding QA Principle: HeeraMon does not need to be perfect to be useful. It must be predictable where predictability matters, transparent where uncertainty exists, and safe when something goes wrong.

13 Document Control

Version	Date	Author / Role	Changes
1.0	September 30, 2026	Product / Engineering	Initial release

Approval: This document defines the testing and QA approach for HeeraMon. All subsequent design and implementation work must remain consistent with the principles stated herein.

HeeraMon flies to help people find job that fits not only their career, but the life they want to live.



Saadman Sakib
Product Engineer